

Is It the Bytes or the Parallelism? A Transport-Ablation Study of Fused-MoE Dispatch Latency

MoE Breakdown Project — Internal Technical Report (expt2 + expt2.5)

Qwen3-30B-A3B · NVIDIA H200 SXM5 · PACE ICE · combined dataset: 1,252 benchmark conditions

Abstract

Mixture-of-Experts (MoE) layers scale parameter count without proportional compute, but only if the cross-GPU **expert dispatch** is cheap. Prior work in this project (expt1) showed that expert-parallel (EP) dispatch dominates fused-MoE latency at large token counts — `tp1-ep4` is $\sim 16\times$ slower than `tp1-ep1` at 65k tokens — even over NVSwitch. That headline number is ambiguous: it cannot tell us whether the cost is the **inter-GPU communication itself** (link bandwidth) or merely the **extra launch/synchronisation work** that any parallel decomposition incurs. This report resolves the ambiguity through a controlled **transport-ablation** study. Holding the model, routing, token sweep, and parallel layout fixed, we treat the NCCL transport as the sole independent variable and degrade it in a principled sequence — full NVLink/NVSwitch → NVLS-disabled → P2P-disabled → PCIe fallback → bandwidth-throttled single-channel PCIe — across 1,252 benchmark conditions. We report four findings. **(1)** The interconnect penalty does **not** turn over at large token counts as we had hypothesised; it rises to $\sim 5\times$ and **plateaus** all the way to 65k tokens — the layer is communication-bound across the entire prefill range for this shape. **(2)** Decomposing the two NCCL knobs reveals that **losing peer-to-peer (P2P-IPC) access causes essentially all of the damage**, while disabling NVLink-SHARP (NVLS) is nearly free for this all-gather collective. **(3)** The slowdown is a **bandwidth cliff, not a channel-count problem**: achieved all-gather bandwidth collapses $\sim 15\text{--}20\times$ from NVLink to PCIe, and raising NCCL channels from 1 to 8 does not recover it. **(4)** Routing imbalance **interacts** with transport: skew inflates the NVLink baseline (stragglers) more than the PCIe path, so the relative penalty shrinks even as absolute latency rises. Together these results give a precise operating map: below ~ 512 tokens MoE dispatch is transport-insensitive (launch/sync-bound); above it, intra-node P2P connectivity is a first-order determinant of latency.

1. Introduction

A fused-MoE layer routes each token to a small set of experts (here, top-8 of 128). Under **expert parallelism**, experts are sharded across GPUs, so every forward pass must **dispatch** tokens to the GPUs holding their experts and **combine** the results — an all-gather/reduce-scatter pattern over NCCL. As batch (token) count grows, this dispatch volume grows with it, and at some point it stops being free.

Expt1 quantified the symptom: at 65,536 tokens, `tp1-ep4` ran $\sim 16\times$ slower than the single-GPU `tp1-ep1`, even on NVSwitch. But a single end-to-end ratio confounds two mechanisms:

- **(H-comm) Communication-bound.** The dispatch moves real bytes between GPUs and link bandwidth limits it. If true, interconnect topology/placement is a first-order lever.

- **(H-sync) Sync/launch-bound.** Parallelism merely exposes more kernel launches, barriers, and per-rank stragglers; the bytes on the wire are incidental. If true, a faster link would not help, and the fix is in scheduling/kernels.

On any single machine the interconnect is a constant, so H-comm and H-sync cannot be separated by observation alone. **Expt2 turned the interconnect into a controlled variable** by degrading the NCCL transport while pinning everything else, establishing the basic effect over a decode→prefill sweep (1–8192 tokens, uniform routing, 3 transports). **Expt2.5 extends that study along four axes** that expt2's analysis flagged as open questions:

1. **Token range to 65,536** — does the penalty ever turn over as GEMMs go compute-bound? (§5.1)
2. **NVLS × P2P factorial** — which NCCL mechanism actually costs the latency? (§5.2)
3. **Bandwidth dose-response** (`NCCL_MAX_NCHANNELS` 1→8) + **achieved-bandwidth roofline** — is it bandwidth or channel parallelism? (§5.3)
4. **Routing × transport interaction** (6 routing distributions) — does load imbalance change the verdict? (§5.4)

Contributions. (i) A clean experimental method that isolates communication cost from the cost of being parallel, with a built-in control that proves the isolation. (ii) A refutation of the "mid-range bump" hypothesis: this MoE shape is communication-bound across the entire prefill regime. (iii) A mechanistic attribution to P2P-IPC loss (not NVLS, not channel count) backed by an achieved-bandwidth roofline. (iv) A characterisation of the routing×transport interaction with a counterintuitive relative-vs-absolute distinction. (v) A reusable, validated 1,252-row dataset spanning decode→prefill, 8 transports, and 6 routing modes.

2. Background

2.1 NCCL intra-node transports

For intra-node collectives on an NVLink/NVSwitch system, NCCL can select among several transports, controlled by environment variables:

Mechanism	Env var to disable	What it provides
NVLS (NVLink SHARP)	<code>NCCL_NVLS_ENABLE=0</code>	In-switch reduction/multicast over NVLink
P2P-IPC (peer GPU access)	<code>NCCL_P2P_DISABLE=1</code>	Direct GPU↔GPU copies over NVLink/PCIe without staging through host
Channels	<code>NCCL_MAX_NCHANNELS=k</code>	Number of parallel ring/tree pipelines

Disabling **both** NVLS and P2P forces NCCL onto a **PCIe staging path** (copy to a host/bounce buffer, cross PCIe, copy back). On our H200 nodes this drops nominal intra-node bandwidth from ~900 GB/s (NVLink/NVSwitch) to ~64 GB/s (PCIe), and a single channel throttles it further.

2.2 The model shape (fixed for all studies)

Qwen3-30B-A3B MoE layer: `hidden=2048`, `intermediate=768`, `num_experts=128`, `top-k=8`, `bf16`, backend `allgather_reducescatter` (standard NCCL, **not** DeepEP). This is a narrow-expert shape: each expert GEMM is small, so the layer is unusually launch/communication-sensitive — a useful stress test for the dispatch path.

3. Methodology

3.1 Design

We benchmark one fixed MoE layer under four **parallel layouts** and vary only the NCCL transport. The layouts are chosen to span the relevant collective patterns:

Layout	World size	Active collectives	Role
tp1-ep1	1	none	Control (no inter-GPU comm)
tp1-ep2	2	EP all-gather	Onset
tp1-ep4	4	EP all-gather (max EP on a 4-GPU node)	Max comm volume
tp2-ep2	4	TP all-reduce + EP all-gather	Stacked collectives

tp1-ep1 is the linchpin of the design: with ep=1 there is no inter-GPU dispatch, so by construction its latency must be identical across all transport conditions and all routing modes. Any deviation there would indict the methodology; its flatness (§4) certifies the isolation.

3.2 The combined dataset

Study	study_name	Transports	Routing	Tokens	Rows
expt2	transport-conditions-qwen3	3	uniform	1-8192 (11)	132
expt2.5-A	transport-extended-qwen3	8	uniform	1-65536 (14)	448
expt2.5-B	routing-sweep-qwen3	2	6 modes	1-65536 (14)	672
				Total	1,252

Both expt2 and expt2.5 ran on the **same hardware** (PACE ICE H200 SXM5, single node, 4 GPUs, nodes atl1-1-03-017/018), with identical 56-column schemas, so cross-study latency comparisons are valid. Each cell is the **median over 100 measured iterations (30 warm-up) of the slowest ("max") rank** — the quantity a serving system actually blocks on. The eight transport conditions form the principled degradation ladder of §2.1.

3.3 Derived metrics

- **Slowdown ratio** = $\text{latency}(\text{degraded}) \div \text{latency}(\text{nvlink_default})$ at the same (layout, tokens, routing). A ratio of 1.0 means "the interconnect is irrelevant here."
- **Achieved all-gather bandwidth** = $\text{allgather_recv_bytes} / \text{network_time}$. We emphasise the **NVLink-to-PCIe ratio**, which is invariant to per-iteration normalisation of the profiler's network bucket.

4. Validation

Before interpreting effects we confirm the experiment is sound. All checks pass:

- **Control invariance (transport)**. Across all 8 transports, tp1-ep1 latency stays within **[0.98, 1.07]** of its NVLink value (mean 1.000). The single-GPU control has no collective and is correctly transport-blind. Its network bucket is exactly **0 ms**.
- **Control invariance (routing)**. At tp1-ep1, latency is flat across 5 of 6 routing modes; the lone exception is worst-case (alpha≈16), which is faster because pinning tokens to ~16 experts

touches less weight memory on-device. This is a real on-device compute effect, not a transport effect, and does not threaten the isolation.

- **Cross-study reproducibility.** On the 132 cells shared by expt2 and expt2.5-A, the **median absolute latency difference is 1.3%** (p95 = 8.8%). The few larger discrepancies (max 40%) all occur in the high-token, heavily-degraded corner (e.g. `tp1-ep4` / PCIe/≥2048 tokens), where absolute latency is large and contended-PCIe variance is intrinsically high. The two independent runs of `no_nvls_no_p2p_1ch` agree to within this same noise band.

5. Results

5.1 RQ1 — The penalty plateaus; it never turns over

Our going-in hypothesis (from expt2) was that interconnect sensitivity would be a bump: ~1.0 at small token counts, peaking in a moderate regime, then **declining toward 1.0** at large token counts as the expert GEMMs became compute-bound and dwarfed the communication. Extending the sweep to 65,536 tokens **refutes this**.

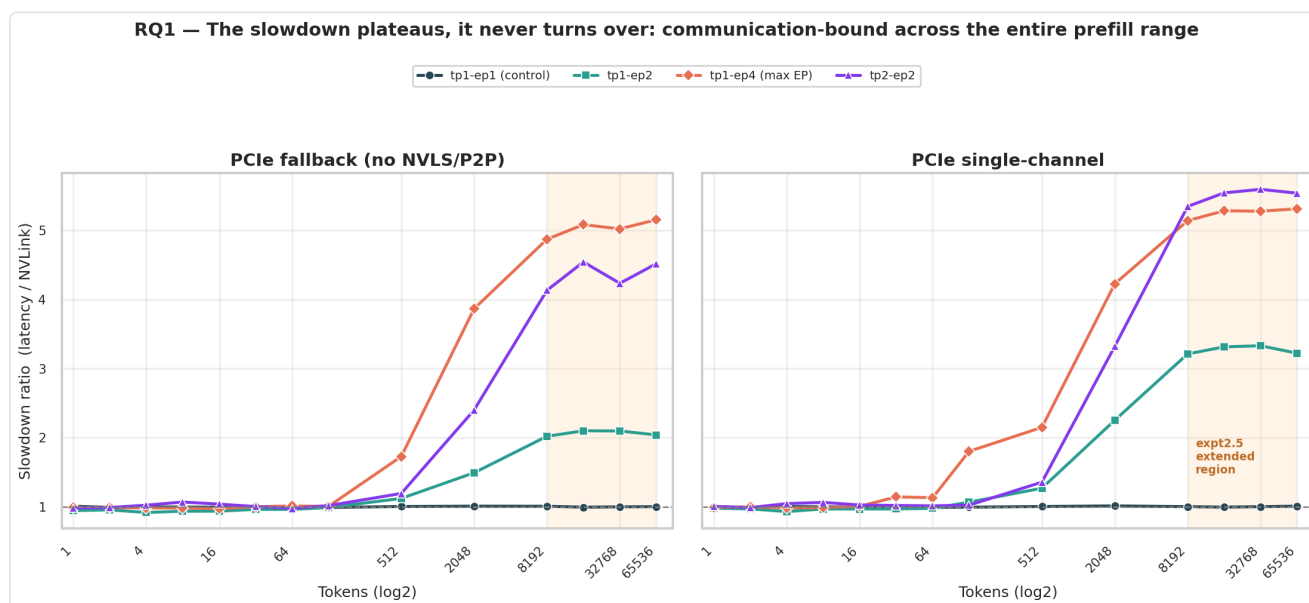


Figure 1. Slowdown ratio vs. token count (to 65k) for the two PCIe conditions; one line per layout. The shaded band marks the expt2.5 extension beyond expt2's 8192-token ceiling. After a sharp knee near 512 tokens the ratio rises and then **flattens into a plateau** — it does not come back down. The control (`tp1-ep1`) hugs 1.0 throughout.

For `tp1-ep4` under full PCIe fallback the ratio is 4.87× at 8192 tokens and **5.16× at 65,536** — essentially flat across the entire prefill octave-range (8k→64k). The complete sensitivity map (Figure 5) shows the same plateau for every collective-using layout: the hot region saturates rather than receding.

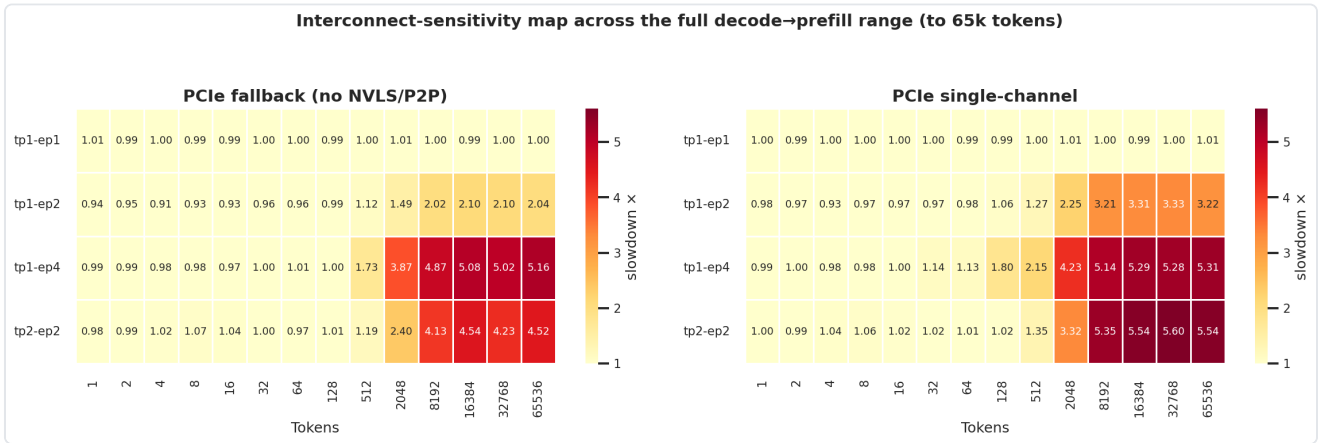


Figure 5. Interconnect-sensitivity map over the full decode→prefill range. The control row (`tp1-ep1`) is uniformly ~1.0. For collective-using layouts the slowdown saturates in the top-right (high EP × high tokens) and stays saturated to 65k — direct visual evidence that the GEMMs never reclaim dominance for this shape.

Interpretation. Profiling confirms the cause: at `tp1-ep4`/65k the GPU-compute bucket is ≈0 ms while network + sync rises from ~45% of the budget on NVLink to ~60% on PCIe. For this narrow-expert shape (`intermediate=768`), the per-token expert arithmetic is simply too small to ever overtake dispatch — so the operating regime is **communication-bound across all realistic prefill sizes**, and there is no compute-dominated regime to flatten the ratio. The hypothesis is updated accordingly.

5.2 RQ2 — It's P2P-IPC, not NVLS

expt2 flipped two NCCL knobs at once (`NVLS_ENABLE=0` and `P2P_DISABLE=1`), so it could not attribute the cost. The 4-cell factorial does.

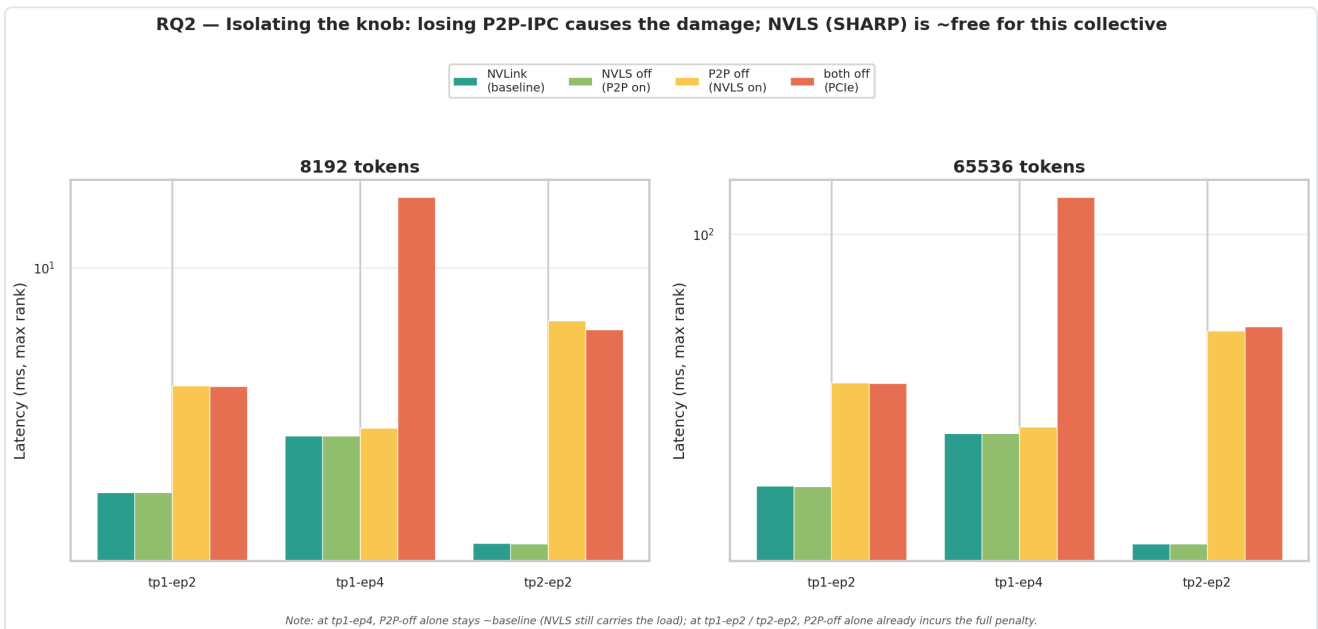


Figure 2. Per-layout latency for the four ablation cells at 8192 and 65,536 tokens (log scale). Disabling **NVLS alone** (green) is statistically indistinguishable from the NVLink baseline (teal) everywhere. The penalty appears only when **P2P-IPC** is lost.

At `tp1-ep4`/65k: `nvls_off` = 1.00x, `p2p_off` = 1.05x, but `no_nvls_no_p2p` = 5.16x. The headline: **NVLink-SHARP contributes essentially nothing to this all-gather/reduce-scatter dispatch;**

the entire interconnect tax is the loss of direct peer GPU access and the resulting fallback to PCIe host-staging.

An important layout-dependent nuance (Figure 2 note): the interaction between the two knobs is not uniform. At `tp1-ep4`, `p2p_off` alone stays near baseline (1.05×) — with P2P off but NVLS on, NCCL still finds a fast NVLink path — and only losing **both** triggers the fallback. But at `tp1-ep2` and `tp2-ep2`, `p2p_off` alone already incurs the full penalty (2.04× and 4.39×). In other words, the second knob is redundant for some collective patterns and necessary for others. The safe operational statement is therefore: **preserving P2P-IPC is sufficient to avoid the penalty in every layout we tested; NVLS is not the lever.**

5.3 RQ3 — A bandwidth cliff, not a channel-count problem

If the penalty were about pipeline parallelism within NCCL, adding channels would recover it. It does not.

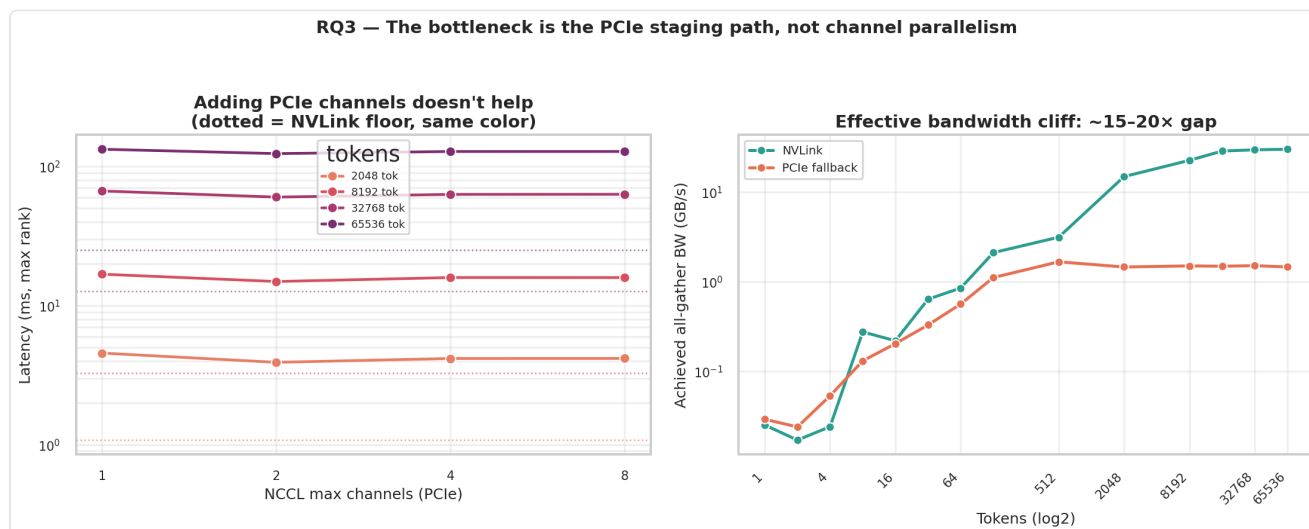


Figure 3. Left: `tp1-ep4` latency vs. `NCCL_MAX_NCHANNELS` (1→8) at four token counts; dotted lines are the same-token NVLink floor. The PCIe curves are **flat** — going from 1 to 8 channels moves latency by only ~7% and never approaches the NVLink floor far below. Right: achieved all-gather bandwidth vs. tokens. Both transports ramp as fixed overheads amortise, then **plateau**: NVLink saturates near ~30 GB/s (profiler wall-clock; the ratio is the robust quantity), PCIe near ~1.5 GB/s.

The NVLink-to-PCIe achieved-bandwidth **ratio is ~16× at 8192 tokens and ~21× at 65k** — invariant to how the profiler normalises its network bucket, and consistent with the ~5× end-to-end latency penalty once the non-communication floor is included. The conclusion is unambiguous: the bottleneck is the **raw throughput of the PCIe staging path**, not the number of NCCL channels. Tuning `NCCL_MAX_NCHANNELS` is not a remedy.

5.4 RQ4 — Routing skew shrinks the relative penalty but raises absolute latency

expt2 deliberately fixed routing to `uniform` to isolate transport. expt2.5-B relaxes that across six distributions to test for interaction.

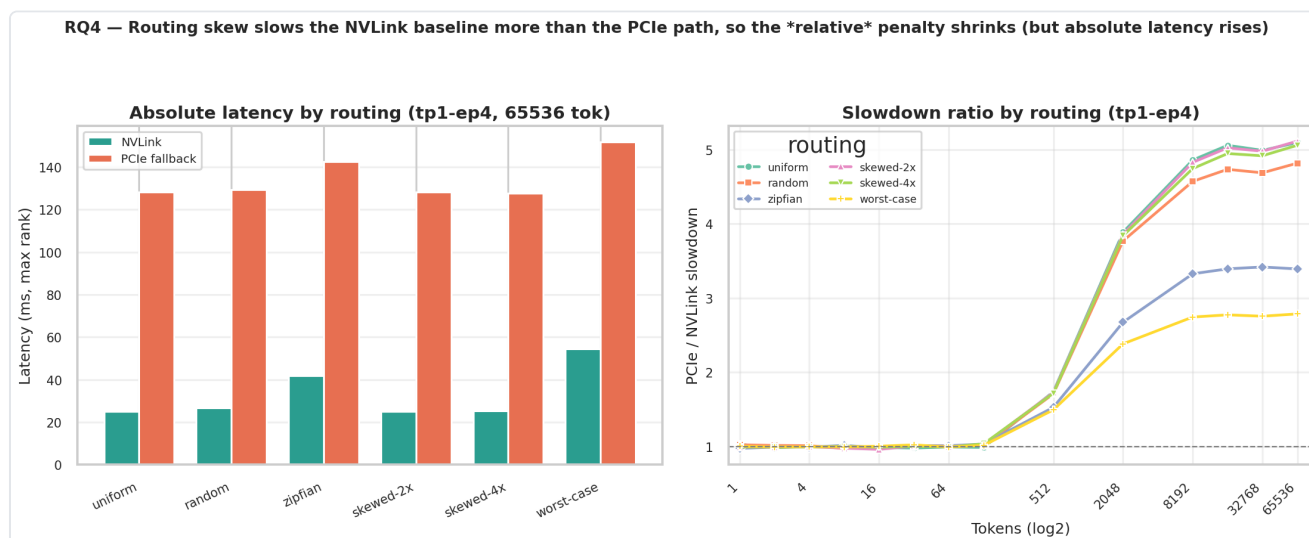


Figure 4. Left: absolute `tp1-ep4` /65k latency by routing mode for NVLink vs. PCIe. Right: PCIe/NVLink slowdown ratio vs. tokens, one line per routing mode.

The interaction is real and initially counterintuitive. Heavily skewed routing (`worst-case`, `zipfian`) produces the **lowest** slowdown ratios (worst-case $\approx 2.8\times$, zipfian $\approx 3.4\times$) versus balanced routing (uniform/random/skewed-2x/4x $\approx 5.1\times$). The mechanism is visible in the left panel: skew creates **straggler ranks** that inflate the NVLink baseline (worst-case NVLink latency is $\sim 2\times$ uniform's), while the PCIe latency is already so dominated by transport that skew adds proportionally less. Since the ratio divides by the (now larger) NVLink baseline, it shrinks.

The practical takeaway is the opposite of the ratio's optimism: in absolute terms, skewed routing is still the **slowest** configuration on PCIe (worst-case 151.8 ms vs. uniform 128.1 ms at 65k). Load imbalance and transport degradation are additive harms; the shrinking ratio is an artefact of normalisation, not a reprieve. Reporting only the slowdown ratio here would be misleading — both views are shown.

6. Discussion: an operating map for MoE serving

Combining the four results yields a compact decision guide for this class of MoE layer:

- Two regimes, one sharp knee (~512 tokens).** Below the knee (decode-scale batches), dispatch is launch/sync-bound and the interconnect is irrelevant — optimise kernels/scheduling, not topology. Above it (prefill-scale batches), the layer is communication-bound and intra-node P2P connectivity is first-order.
- The plateau is the worst case, and it arrives early.** The penalty saturates near $\sim 5\times$ by 8k tokens and stays there to 65k. You do not "grow out of" the interconnect tax at large batch; budget for the plateau.
- Protect P2P-IPC above all.** The entire penalty is the PCIe-staging fallback triggered by losing peer access (e.g. via restrictive GPU isolation/cgroups, MIG, or `--gpus-per-task=1`-style scheduling). NVLS and channel tuning are second-order. This is directly actionable for cluster/SLURM configuration.
- Imbalance and slow transport stack.** Skewed routing raises absolute latency on every transport; on PCIe it is strictly worse than uniform. Do not be fooled by its smaller ratio.

7. Threats to validity

- **Profiler attribution.** The per-bucket profiler attributes ≈ 0 ms to the GEMM/ `gpu_compute` bucket for this narrow-expert shape. We therefore use buckets only for **proportional** comm-vs-sync attribution and for **bandwidth ratios** (normalisation-invariant), never as absolute per-iteration compute timing. End-to-end latency (the primary metric) is unaffected.
 - **Single shape.** All results are for Qwen3-30B-A3B (`intermediate=768`). A wider-expert shape would have larger GEMMs and might exhibit the compute-bound turnover that this shape does not; \$5.1's "no turnover" claim is shape-specific. Cross-shape replication is the obvious next study.
 - **Single node, intra-node only.** Everything is single-node H200 over NVLink/NVSwitch vs. PCIe. Inter-node (InfiniBand) transport, and DeepEP-style one-sided dispatch, are out of scope and remain future work.
 - **High-token PCIe variance.** Run-to-run variance is larger in the saturated PCIe corner (§4); we mitigate with 100-iteration medians and confirm cross-study agreement within that band.
-

8. Related work (within the project)

This report builds directly on **expt1** (fused-MoE latency characterisation: routing, tokens, parallelism, backends; source of the $\sim 16\times$ -at-65k motivating result) and **expt2** (the initial 3-transport ablation over 1-8192 tokens that established the basic effect and posed the four questions answered here). **expt2.5** is the extension; this document presents **expt2** and **expt2.5** as a single coherent study.

9. Conclusion

By turning the NCCL transport into a controlled variable, we separated the cost of communication from the cost of being parallel in fused-MoE dispatch. For Qwen3-30B-A3B on H200, the dispatch is **communication-bound across the entire prefill range** — the interconnect penalty rises to $\sim 5\times$ and plateaus to 65k tokens rather than receding. The penalty is mechanistically the loss of **P2P-IPC** (not NVLS, not channel count), manifesting as a $\sim 15\text{--}20\times$ achieved-bandwidth cliff on the PCIe staging path. Routing skew compounds absolute latency on every transport even as it deflates the slowdown ratio. The net guidance for MoE serving on multi-GPU nodes is concrete: above ~ 512 tokens, **preserve direct peer-GPU connectivity** — it is worth a $5\times$ latency factor that no amount of channel tuning or larger batching will recover.

Appendix: Reproducibility

- **Data:** `expt2/all_runs.zip` (jobs 5440143–5440154, 132 rows) + `expt2.5/all_runs.zip` (jobs 5442861–5442900, 1,120 rows), consolidated into `combined.csv` (1,252 rows).
- **Primary metric:** `latency_median_ms_max_rank` (median of 100 iters, slowest rank). Slowdown = condition $\div$ `nvlink_default` at matched (`layout`, `tokens`, `routing`).
- **Transport ladder:** `nvlink_default`, `nvls_off`, `p2p_off`, `no_nvls_no_p2p`, `no_nvls_no_p2p_{8,4,2,1}ch` (see §2.1 for env vars).
- **Hardware:** PACE ICE H200 SXM5, single node, 4 GPUs, nodes `atl1-1-03-017/018`.
- **Figures:** regenerated with Seaborn/Matplotlib via `figs_a.py` + `figs_b.py`; analysis in `findings.py`, `checks2.py`, `explore*.py`.